OPERATING SYSTEM (315319)

Question Bank for Practical Exam – Solutions

THEORY QUESTIONS

1) List any four system calls and write their use.

System Call	Use
`fork()`	Creates a new (child) process
`exec()`	Replaces the current program with a new program
`getpid()`	Returns the process ID of the running process
`exit()`	Terminates the process; OS reclaims its resources

Other calls: `pipe()` (IPC), `open()`, `close()`, `stat()`.

2) Write system calls related to process management.

- `fork()` – create a process
- `exec()` family – load a new program into a process
- `wait()` – parent waits for child to finish
- `exit()` – terminate a process
- `getpid()`, `getppid()` – get process ID / parent ID

3) What is the difference between sleep and wait?

sleep	wait
Pauses a process for a fixed time	Parent waits until a child process terminates
Time-based	Event-based
Resumes automatically after the time ends	Resumes when child exits; collects exit status
Example: `sleep 30`	Example: `wait()` after `fork()`

4) Write signal names with signal numbers used in the kill command.

Signal	No.	Meaning
SIGHUP	1	Hang up
SIGINT	2	Interrupt (Ctrl+C)
SIGQUIT	3	Quit
SIGKILL	9	Forceful termination (cannot be caught)
SIGTERM	15	Graceful termination
SIGCONT	18	Continue a stopped process
SIGSTOP	19	Stop (pause) a process

Use `kill -l` to list all signals. Example: `kill -9 1234`.

5) What are the various options of the ps command?

Command	Description
`ps`	Processes of the current terminal
`ps -f`	Full-format listing (UID, PID, PPID, C, STIME, TTY, TIME, CMD)
`ps -u username`	Processes of a particular user
`ps -e`	All processes on the system
`ps -ef`	All processes in full format
`ps -l`	Long format (shows PRI and NI values)

6) Write a command to append the contents of one file to another using cat.

```
cat file1.txt >> file2.txt
```

`>>` appends the content of file1.txt at the end of file2.txt. (`>` would overwrite file2.txt.)

7) Explain the difference between preemptive and non-preemptive SJF.

Non-preemptive SJF	Preemptive SJF (SRTF)
Process keeps the CPU until it completes	CPU can be taken away from the running process
Shortest burst time among arrived processes is picked	Preempted if a new process has a shorter remaining time
Fewer context switches	More context switches
Higher average waiting time	Lower average waiting time

8) What is priority CPU scheduling?

Each process is assigned a priority and the CPU is given to the process with the highest priority. Processes with equal priority are served in FCFS order. It can be preemptive or non-preemptive.

- **Problem:** starvation of low-priority processes.
- **Solution:** aging – gradually increase the priority of waiting processes.

9) How does context switching occur in Round Robin scheduling and what is its impact on system performance?

Each process gets a fixed time quantum. When the quantum expires:

- A timer interrupt occurs.
- OS saves the state of the running process in its PCB.
- The process is moved to the tail of the ready queue.
- State of the next process is loaded and it starts running.

Impact:

- Small quantum – good response time, but too many context switches, so more overhead.
- Large quantum – fewer switches, but behaves like FCFS and response time becomes poor.

10) What is Banker's algorithm?

A deadlock **avoidance** algorithm. A resource request is granted only if the system remains in a safe state.

Data structures: Available, Max, Allocation, Need (Need = Max − Allocation).

Safety algorithm:

- Set Work = Available.
- Find a process with Need ≤ Work that is not finished.
- Assume it finishes: Work = Work + Allocation. Mark it finished.
- Repeat. If all processes finish, the system is safe and the order is the safe sequence.

SUMS SOLVING

CPU Scheduling Algorithms | Page Replacement Algorithms | Banker's Algorithm

Formulas to remember

Term	Meaning / Formula
AT	Arrival Time
BT	Burst Time (CPU time needed)
CT	Completion Time (read from Gantt chart)
TAT	Turnaround Time = CT - AT
WT	Waiting Time = TAT - BT
A(TAT), A(WT)	Average = sum of all values / number of processes
Page hit ratio	Hits / total references
Page miss ratio	Page faults / total references

PART A – CPU SCHEDULING ALGORITHMS

Q.1 First Come First Served (FCFS)

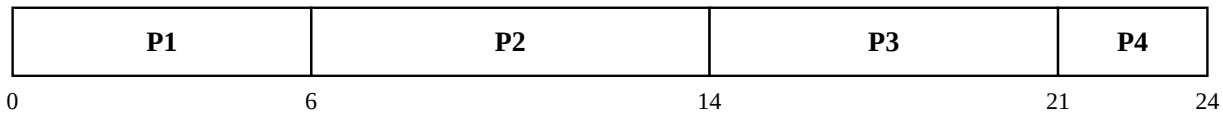
Calculate A(TAT) and A(WT).

Process	AT	BT
P1	0	6
P2	1	8
P3	2	7
P4	3	3

Find: A(WT), A(TAT)

Rule: The process that arrives first gets the CPU first (non-preemptive).

Gantt chart:



Process	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	8	14	13	5
P3	2	7	21	19	12
P4	3	3	24	21	18

CT = completion time, **TAT = CT - AT**, **WT = TAT - BT**

A(WT) = (0 + 5 + 12 + 18) / 4 = 35 / 4 = 8.75 ms

A(TAT) = (6 + 13 + 19 + 21) / 4 = 59 / 4 = 14.75 ms

Q.2 Shortest Job First (SJF) - Non-preemption

Calculate A(TAT) and A(WT).

Process	AT	BT
P1	0	6
P2	1	8
P3	2	7
P4	3	3

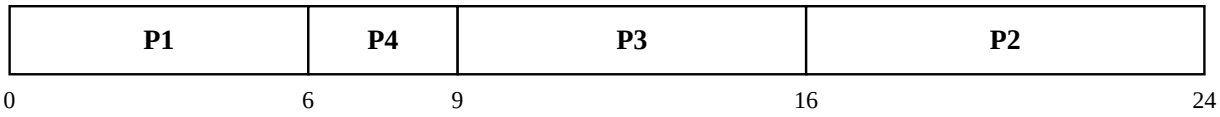
Find: A(WT), A(TAT)

Rule: Among arrived processes, the one with the smallest burst time runs to completion.

Steps:

At t = 0: ready = P1(6) -> **P1** (shortest burst time)
At t = 6: ready = P2(8), P3(7), P4(3) -> **P4** (shortest burst time)
At t = 9: ready = P2(8), P3(7) -> **P3** (shortest burst time)
At t = 16: ready = P2(8) -> **P2** (shortest burst time)

Gantt chart:



Process	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	8	24	23	15
P3	2	7	16	14	7
P4	3	3	9	6	3

CT = completion time, **TAT = CT - AT**, **WT = TAT - BT**

A(WT) = (0 + 15 + 7 + 3) / 4 = 25 / 4 = 6.25 ms

A(TAT) = (6 + 23 + 14 + 6) / 4 = 49 / 4 = 12.25 ms

Q.3 Shortest Job First (SJF) - Preemption (SRTF)

Calculate A(TAT) and A(WT).

Process	AT	BT
P1	0	6
P2	1	8
P3	2	7
P4	3	3

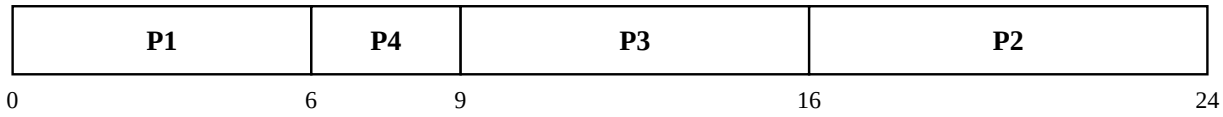
Find: A(WT), A(TAT)

Rule: Process with the shortest remaining time runs. A new arrival with a shorter remaining time preempts the running process.

Steps:

At t = 0: ready = P1(6) -> **P1** (shortest remaining time)
At t = 6: ready = P2(8), P3(7), P4(3) -> **P4** (shortest remaining time)
At t = 9: ready = P2(8), P3(7) -> **P3** (shortest remaining time)
At t = 16: ready = P2(8) -> **P2** (shortest remaining time)

Gantt chart:



Process	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	8	24	23	15
P3	2	7	16	14	7
P4	3	3	9	6	3

CT = completion time, **TAT = CT - AT**, **WT = TAT - BT**

A(WT) = (0 + 15 + 7 + 3) / 4 = 25 / 4 = 6.25 ms

A(TAT) = (6 + 23 + 14 + 6) / 4 = 49 / 4 = 12.25 ms

Q.4 Priority Scheduling - Non-preemption

Calculate A(TAT) and A(WT).

Process	AT	BT	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

Find: A(WT), A(TAT)

Rule: Lower priority number = higher priority. Running process is not interrupted.

Steps:

At t = 0: ready = P1(pr 3) -> **P1** (highest priority)
At t = 8: ready = P2(pr 1), P3(pr 4), P4(pr 2) -> **P2** (highest priority)
At t = 12: ready = P3(pr 4), P4(pr 2) -> **P4** (highest priority)
At t = 17: ready = P3(pr 4) -> **P3** (highest priority)

Gantt chart:

Process	AT	BT	Pr	CT	TAT	WT
P1	0	8	3	8	8	0
P2	1	4	1	12	11	7
P3	2	9	4	26	24	15
P4	3	5	2	17	14	9

CT = completion time, **TAT = CT - AT**, **WT = TAT - BT**

A(WT) = (0 + 7 + 15 + 9) / 4 = 31 / 4 = 7.75 ms

A(TAT) = (8 + 11 + 24 + 14) / 4 = 57 / 4 = 14.25 ms

Q.5 Priority Scheduling - Preemption

Calculate A(TAT) and A(WT).

Process	AT	BT	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

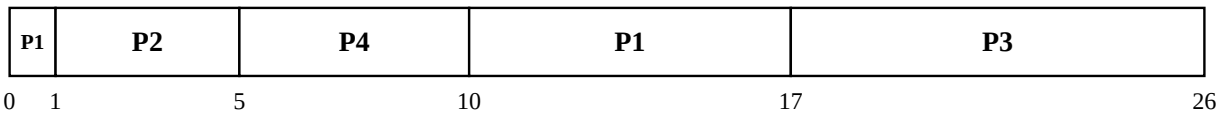
Find: A(WT), A(TAT)

Rule: Lower priority number = higher priority. A higher-priority arrival preempts the running process.

Steps:

At t = 0: ready = P1(pr 3) -> **P1** (highest priority)
At t = 1: ready = P1(pr 3), P2(pr 1) -> **P2** (highest priority)
At t = 5: ready = P1(pr 3), P3(pr 4), P4(pr 2) -> **P4** (highest priority)
At t = 10: ready = P1(pr 3), P3(pr 4) -> **P1** (highest priority)
At t = 17: ready = P3(pr 4) -> **P3** (highest priority)

Gantt chart:



Process	AT	BT	Pr	CT	TAT	WT
P1	0	8	3	17	17	9
P2	1	4	1	5	4	0
P3	2	9	4	26	24	15
P4	3	5	2	10	7	2

CT = completion time, **TAT = CT - AT**, **WT = TAT - BT**

A(WT) = (9 + 0 + 15 + 2) / 4 = 26 / 4 = 6.5 ms

A(TAT) = (17 + 4 + 24 + 7) / 4 = 52 / 4 = 13 ms

Q.6 Round Robin (RR)

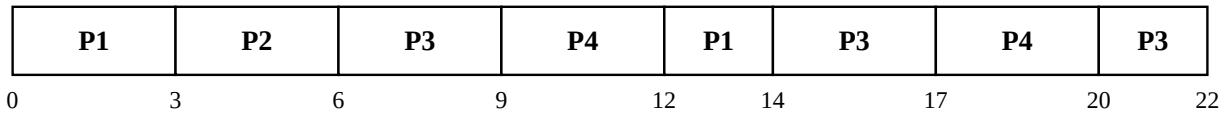
Calculate A(TAT) and A(WT). Time Quantum = 3 ms.

Process	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Find: A(WT), A(TAT)

Rule: Each process runs for at most one time quantum, then goes to the end of the ready queue. New arrivals join the queue before a preempted process.

Gantt chart:



Process	AT	BT	CT	TAT	WT
P1	0	5	14	14	9
P2	1	3	6	5	2
P3	2	8	22	20	12
P4	3	6	20	17	11

CT = completion time, **TAT = CT - AT**, **WT = TAT - BT**

A(WT) = (9 + 2 + 12 + 11) / 4 = 34 / 4 = 8.5 ms

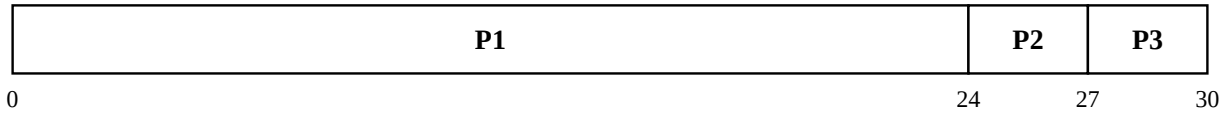
A(TAT) = (14 + 5 + 20 + 17) / 4 = 56 / 4 = 14 ms

Q.7 All processes arrive at time 0 (FCFS, SJF and RR)

Calculate A(WT) and A(TAT) for FCFS, SJF and Round Robin (TQ = 4 ms).

Process	AT	BT
P1	0	24
P2	0	3
P3	0	3

(a) FCFS

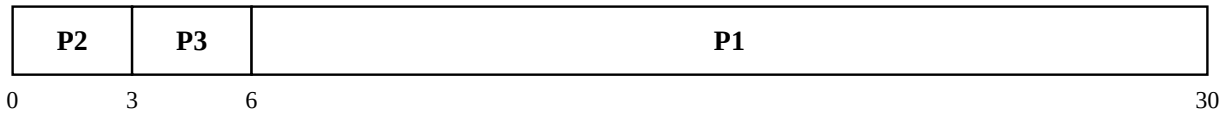


Process	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	0	3	27	27	24
P3	0	3	30	30	27

A(WT) = (0 + 24 + 27) / 3 = 51 / 3 = 17 ms

A(TAT) = (24 + 27 + 30) / 3 = 81 / 3 = 27 ms

(b) SJF (non-preemptive)

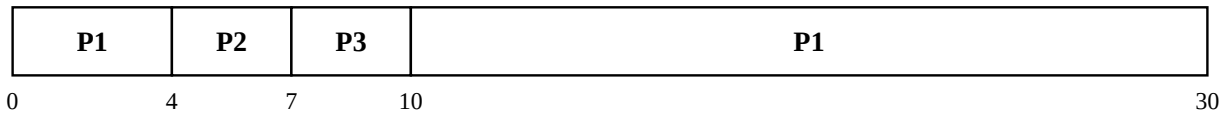


Process	AT	BT	CT	TAT	WT
P1	0	24	30	30	6
P2	0	3	3	3	0
P3	0	3	6	6	3

A(WT) = (6 + 0 + 3) / 3 = 9 / 3 = 3 ms

A(TAT) = (30 + 3 + 6) / 3 = 39 / 3 = 13 ms

(c) Round Robin, TQ = 4



Process	AT	BT	CT	TAT	WT
P1	0	24	30	30	6
P2	0	3	7	7	4
P3	0	3	10	10	7

A(WT) = (6 + 4 + 7) / 3 = 17 / 3 = 5.67 ms

A(TAT) = (30 + 7 + 10) / 3 = 47 / 3 = 15.67 ms

Conclusion:

Algorithm	A(WT) ms	A(TAT) ms
FCFS	17	27
SJF (non-preemptive)	3	13
Round Robin, TQ = 4	5.67	15.67

SJF gives the minimum average waiting time. FCFS suffers from the convoy effect when a long job comes first.

PART B – PAGE REPLACEMENT ALGORITHMS

Q.8 FIFO Page Replacement

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 | Number of frames = 3

Find: number of page faults, hit ratio and miss ratio

Rule: Replace the page that has been in memory the longest.

Ref.	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame 2		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame 3			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
H / F	F	F	F	F	H	F	F	F	F	F	F	H	H	F	F	H	H	F	F	F

F = page fault (shaded), H = page hit

Total page faults = 15

Total page hits = 5

Hit ratio = 5 / 20 = 0.25 = 25%

Miss ratio = 15 / 20 = 0.75 = 75%

Q.9 LRU Page Replacement

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 | Number of frames = 3

Find: number of page faults, hit ratio and miss ratio

Rule: Replace the page that has not been used for the longest time.

Ref.	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame 2		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame 3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
H / F	F	F	F	F	H	F	H	F	F	F	F	H	H	F	H	F	H	F	H	H

F = page fault (shaded), H = page hit

Total page faults = 12

Total page hits = 8

Hit ratio = 8 / 20 = 0.4 = 40%

Miss ratio = 12 / 20 = 0.6 = 60%

Q.10 Optimal Page Replacement

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 | Number of frames = 3

Find: number of page faults, hit ratio and miss ratio

Rule: Replace the page that will not be used for the longest time in the future.

Ref.	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Frame 2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
Frame 3			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
H / F	**F**	**F**	**F**	**F**	**H**	**F**	**H**	**F**	**H**	**H**	**F**	**H**	**H**	**F**	**H**	**H**	**H**	**F**	**H**	**H**

F = page fault (shaded), H = page hit

Total page faults = 9

Total page hits = 11

Hit ratio = 11 / 20 = 0.55 = 55%

Miss ratio = 9 / 20 = 0.45 = 45%

Comparison

Algorithm	Page faults	Hit ratio
FIFO	15	5/20 = 25%
LRU	12	8/20 = 40%
Optimal	9	11/20 = 55%

Optimal gives the fewest faults but needs future knowledge, so it is used only as a benchmark.

PART C – BANKER'S ALGORITHM (DEADLOCK AVOIDANCE)

Q.11 Find the Need matrix and check whether the system is in a safe state.

5 processes (P0 to P4) and 3 resource types (A, B, C). Available = (3, 3, 2).

Process	Allocation			Max			Need = Max - Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

Step 1: Need = Max - Allocation (already filled in the table).

Step 2 (Safety algorithm): Work = Available = (3, 3, 2). Pick a process whose Need <= Work, assume it finishes and add its Allocation to Work.

Process	Need	Work (before)	Work (after = Work + Alloc)
P1	(1, 2, 2)	(3, 3, 2)	(5, 3, 2)
P3	(0, 1, 1)	(5, 3, 2)	(7, 4, 3)
P4	(4, 3, 1)	(7, 4, 3)	(7, 4, 5)
P0	(7, 4, 3)	(7, 4, 5)	(7, 5, 5)
P2	(6, 0, 0)	(7, 5, 5)	(10, 5, 7)

Safe sequence: < P1, P3, P4, P0, P2 >

All processes can finish, so the system is in a safe state.

Q.12 P1 requests (1, 0, 2). Can the request be granted immediately?

Step 1: Request <= Need of P1? (1, 0, 2) <= (1, 2, 2) Yes

Step 2: Request <= Available? (1, 0, 2) <= (3, 3, 2) Yes

Step 3: Pretend to allocate:

Available = (3, 3, 2) - (1, 0, 2) = (2, 3, 0)
Allocation of P1 = (2, 0, 0) + (1, 0, 2) = (3, 0, 2)
Need of P1 = (1, 2, 2) - (1, 0, 2) = (0, 2, 0)

Step 4: Run the safety algorithm with Work = (2, 3, 0):

Process	Need	Work (before)	Work (after = Work + Alloc)
P1	(0, 2, 0)	(2, 3, 0)	(5, 3, 2)
P3	(0, 1, 1)	(5, 3, 2)	(7, 4, 3)
P4	(4, 3, 1)	(7, 4, 3)	(7, 4, 5)
P0	(7, 4, 3)	(7, 4, 5)	(7, 5, 5)
P2	(6, 0, 0)	(7, 5, 5)	(10, 5, 7)

Safe sequence: < P1, P3, P4, P0, P2 >

The new state is safe, so the request of P1 can be granted.

COMMANDS EXECUTION

Part 1: Process related Linux commands

1) top

Purpose: Displays a real-time view of running processes, including CPU usage, memory usage and system information.

Syntax: `top [option]`

Command	Description
`top -h`	Help: displays the help information for top
`top -V`	Version: displays the version of top installed on the system
`top -b`	Batch mode: output can be redirected to a file or used by another command instead of refreshing interactively
`top -b -n 1`	-b = batch mode, -n 1 = only one iteration
`top -H`	Show threads: displays individual threads instead of only processes
`top -i`	Ignore idle processes: does not display idle or inactive processes
`top -d 2`	Refresh every 2 seconds: changes the refresh delay between updates
`top -n 5`	Run 5 iterations: updates the display 5 times and then exits
`top -p PID`	Monitor a specific process using its process ID

2) ps

Purpose: ps means Process Status. It displays information about currently running processes.

Syntax: `ps [options]`

Command	Description
`ps`	Current processes: processes associated with the current terminal
`ps -f`	Full format listing
`ps -u username`	Processes of a particular user, e.g. `ps -u student`
`ps -e`	All processes running on the system
`ps -ef`	All processes in full format, with detailed information

Output of ps:

```
PID    TTY     TIME      CMD
2451   pts/0   00:00:00  bash
2784   pts/0   00:00:00  ps
```

This means the bash shell has PID 2451.

Columns of ps -f:

Column	Meaning
UID	User ID
PID	Process ID
PPID	Parent Process ID
C	CPU utilization value
STIME	Start time of the process

Column	Meaning
TTY	Terminal
TIME	CPU time consumed
CMD	Command used to start the process

3) kill

Purpose: Used to terminate a running process manually.

Syntax: `kill [signal] PID`

Command	Description
`kill 1234`	Terminate the process with PID 1234 (verify using `ps -p 1234`)
`kill -9 1234`	Force termination: SIGKILL (9)
`kill -15 1234`	Graceful termination: SIGTERM (15)
`kill 120 230 234`	Terminate multiple processes by specifying several PIDs

Note: Use `kill -9` only when normal termination does not work, because the process does not get an opportunity to clean up its resources.

4) sleep

Purpose: Pause execution for a specified amount of time.

Syntax: `sleep NUMBER[SUFFIX]`

Command	Description
`sleep 30`	Wait for 30 seconds
`sleep 5m`	Wait for 5 minutes
`sleep 2h`	Wait for 2 hours

5) nice

Purpose: Starts a new process with a specified nice value (priority). A higher nice value means lower scheduling priority.

Syntax: `nice -n NICE_VALUE command`

Example: `nice -n 10 sleep 100`

nice = starts a process with the specified nice value; -n 10 = sets the nice value to 10; sleep 100 = keeps the process running for 100 seconds. So the sleep process runs with a nice value of 10.

Checking the nice value:

```
$ nice -n 10 sleep 100 &
[1] 3250
$ ps -l -p 3250
F S UID PID PPID C PRI NI ADDR ...
...                     10
```

Here [1] is the job number and 3250 is the PID of the sleep process. The NI column shows 10, which confirms the process is running with a nice value of 10.

6) renice

Purpose: Changes the nice value (priority) of an already running process.

Syntax: `renice NICE_VALUE -p PID`

Example: `renice 5 -p 1234` changes the nice value of process 1234 to 5.

nice starts a new process with a priority; **renice** changes the priority of a running process.

Part 2: User communication and IPC commands

1) who

Purpose: Displays information about users who are currently logged in.

Syntax: `who [options] [filename]` **Example:** `$ who`

Option	Description
`who -a`	Detailed / all information
`who -b`	Last system boot
`who -d`	Dead processes
`who -H`	Column headings
`who -l`	Login processes
`who -m`	Current terminal / user
`who -p`	Active processes
`who -q`	Logged in users and count
`who -T`	Message status

2) cat

Purpose: Used to create, display and concatenate files.

Task	Command
Create a file	`cat > sample.txt` , type `Hello` and press Ctrl+D
Display file contents	`cat sample.txt`
Display multiple files	`cat file1.txt file2.txt`
Combine files	`cat file1.txt file2.txt > combined.txt`
Append one file to another	`cat file1.txt >> file2.txt`

3) wall

Purpose: Sends (broadcasts) a message to all currently logged in users.

Syntax: `wall [options] [message | file]`

Example: `$ wall "System will restart soon"`

Send a message from a file:

Step	Action
1	Create a file: `nano message.txt`
2	Write the message: *System maintenance will start at 10 PM. Please save your work.*
3	Save the file (Ctrl+O, Enter, Ctrl+X) and then execute: `wall message.txt`

4) write

Purpose: Sends a message to one specific logged in user through the terminal.

Syntax: `write username` (the username can be seen using `who`)

Example: suppose the user *student* is currently logged in.

```
$ write student
Hello            (type the message)
Ctrl+D           (end of input)
```

5) mesg

Purpose: Controls whether other users are permitted to send messages to your terminal using commands such as write.

Syntax: `mesg [y | n]`

Command	Description
`mesg y`	Allows messages: other users can use the write command to send messages
`mesg n`	Blocks messages

6) ipcs

Purpose: Displays information about Inter-Process Communication (IPC) resources related to shared memory, message queues and semaphores.

Syntax: `ipcs [options]`

Command	Description
`ipcs -q`	Message queues: information about active message queues
`ipcs -m`	Shared memory: information about shared memory segments
`ipcs -s`	Semaphore arrays: information about semaphore arrays

Note: Semaphores are used in process synchronization to control access to shared resources.

Part 3: System call programs (C)

How to save, compile and run a C program in Linux

Step	Action
1	Type `nano fork.c` and press Enter
2	Enter the program
3	Save the file: press Ctrl+O, press Enter, press Ctrl+X
4	Compile the program: `gcc fork.c -o fork`
5	Execute the program: `./fork`

1) fork()

Purpose: Creates a new process called the child process. The process that calls fork() is the parent process. Both continue execution from the statement immediately after fork().

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();
    printf("Hello\n");
    return 0;
}
```

Line	Explanation
`stdio.h`	Standard Input/Output library for C
`unistd.h`	Header file provided by Unix/Linux OS
`int main()`	Starting point of the C program
`fork();`	Used to create a new process
`printf("Hello\n");`	Since there are 2 processes, printf is executed by both
`return 0;`	Terminates the program

Before fork(): Parent Process -> fork() **After fork():** Parent Process and Child Process, both continue after fork().

Output: `Hello` is printed twice.

2) exec()

Purpose: Replaces the program currently running in a process with a new program. It does not create a new process by itself.

Syntax: `execl(path, arg0, arg1, ..., NULL);`

```
#include <unistd.h>

int main()
{
    execl("/bin/ls", "ls", "-l", NULL);
    return 0;
}
```

Part	Explanation
`execl()`	Executes another program and replaces the current process
`/bin/ls`	Path of the ls program
`ls`	Name of the program being executed
`-l`	Option that tells ls to display files in long format
`NULL`	Indicates the end of arguments

Output: the current program is replaced with the `ls -l` program, so the files are displayed in long format.

3) getpid()

Purpose: Returns the Process ID (PID) of the currently executing process.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("PID = %d\n", getpid());
    return 0;
}
```

Output (example): `PID = 17183`

4) pipe()

Purpose: Creates a pipe for Inter-Process Communication (IPC). One process can write data into the pipe and another can read it.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int fd[2];
    pipe(fd);
    printf("Pipe created successfully\n");
    return 0;
}
```

Part	Explanation
`int fd[2];`	Creates an integer array with 2 elements to store the two ends of the pipe
`fd[0]`	Read end
`fd[1]`	Write end
`pipe(fd);`	Creates a pipe and stores its file descriptors in fd

Data flow: `fd[1]` (write) -> PIPE -> (read) `fd[0]`. One process writes data through fd[1] and another reads it through fd[0].

5) exit()

Purpose: Terminates the currently running process. After termination, the OS reclaims the resources used by that process.

Syntax: `exit(status);` **Example:** `exit(0);` (0 normally indicates successful termination)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("Program is running...\n");
    printf("Program is terminating...\n");
    exit(0);
    printf("This line will not execute.\n");
    return 0;
}
```

`stdlib.h` includes the declaration of the exit() function. The last printf does not execute because exit(0) terminates the program before reaching that statement.

Output:

```
Program is running...
Program is terminating...
```

6) open() and 7) close()

open() Purpose: Opens a file so that operations such as reading or writing can be performed. **Syntax:** `open(pathname, flags);` Example: `int fd = open("file.txt", O_RDONLY);` (read-only mode).

close() Purpose: Closes an opened file / file descriptor.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    fd = open("sample.txt", O_CREAT | O_WRONLY, 0644);

    if (fd == -1)
    {
        printf("File opening failed.\n");
        return 1;
    }

    printf("File opened successfully.\n");
    printf("File descriptor = %d\n", fd);

    close(fd);
    printf("File closed successfully.\n");
    return 0;
}
```

Part	Explanation
`fcntl.h`	Provides open() function and file opening flags such as O_CREAT and O_WRONLY
`unistd.h`	Provides the close() system call
`int fd;`	Creates a variable to store the file descriptor returned by open()
`O_CREAT`	Create a file if it doesn't exist
`O_WRONLY`	Open file for writing
`0644`	Sets file permission when the file is created
`close(fd);`	Closes the opened file descriptor

Output:

```
File opened successfully.
File descriptor = 3
File closed successfully.
```

Breaking down 0644 (rw-r--r--)

The first 0 indicates that the number is written in octal (base 8). The next three digits give the permission for **owner** (6), **group** (4) and **others** (4).

Number	Permission
4	Read (r)
2	Write (w)
1	Execute (x)
0	No permission

Example: 6 = 4 + 2 = Read + Write = rw-. So 0644 = rw- (owner), r-- (group), r-- (others).

8) stat()

Purpose: Displays information / attributes about a file or directory, such as its size, permissions, timestamps and file type.

```
#include <stdio.h>
#include <sys/stat.h>

int main()
{
    struct stat fileInfo;

    if (stat("sample.txt", &fileInfo) == 0)
    {
        printf("File Information:\n");
        printf("File size: %ld bytes\n", fileInfo.st_size);
        printf("File permission: %o\n", fileInfo.st_mode & 0777);
    }
    else
    {
        printf("Unable to get file information.\n");
    }
    return 0;
}
```

Part	Explanation
`sys/stat.h`	Header file required for the stat() system call and struct stat
`struct stat fileInfo;`	Creates a structure named fileInfo; it stores file size, permissions, file type and timestamps
`&fileInfo`	Address of fileInfo is passed so that stat() can fill the structure with file information
`== 0`	stat() returns 0 on success and -1 on error, so this checks whether the operation was successful
`st_size`	Contains the file size in bytes
`st_mode & 0777`	st_mode contains the file mode and permission info; & 0777 extracts the permission bits (owner, group, others)

Output (example):

```
File Information:
File size: 6 bytes
File permission: 644
```

Tip: create sample.txt first (using `cat > sample.txt` or the open() program), otherwise stat() fails.

Quick summary of system calls

System call	Header file	Purpose
`fork()`	`unistd.h`	Create a child process
`exec()`	`unistd.h`	Replace the current program with a new one
`getpid()`	`unistd.h`	Get the process ID
`pipe()`	`unistd.h`	Create a pipe for IPC
`exit()`	`stdlib.h`	Terminate the process
`open()`	`fcntl.h`	Open (or create) a file
`close()`	`unistd.h`	Close a file descriptor
`stat()`	`sys/stat.h`	Get file information